

Industrial Project

2026-03-31

# AI-Powered Pole Validation

AI-powered pole validation using GIS, imagery, OCR, and multi-system data reconciliation.



## Project Snapshot

### DATE

2026-03-31

### CATEGORY

Industrial Project

### SHORT DESCRIPTION

AI-powered pole validation using GIS, imagery, OCR, and multi-system data reconciliation.

### TAGS / TECH STACK

AI

Computer Vision

YOLO

GIS

OCR

Pole Validation

Infrastructure

## Project Links

[Demo Video](#)

[Demo Video](#)

[GitHub Repository](#)

# Full Project Content

Building an AI system that doesn't just **detect poles** — but **validates and decides**.

GitHub Repository: [dranubhaparashar/Pole-Detection](https://github.com/dranubhaparashar/Pole-Detection)

## Vision

Modern infrastructure validation is slow, manual, and error-prone.

This project introduces an **AI-powered validation framework** that:

- Detects poles
- Verifies ownership
- Assesses structure
- Provides decision-ready outputs

## Project Attributes

| Attribute          | Description                                                                                                                |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| problem-statement  | Manual pole validation is slow, fragmented, and dependent on multiple disconnected systems.                                |
| primary-objective  | Build an AI-assisted framework to detect poles, validate ownership, assess suitability, and support decision-making.       |
| core-technologies  | YOLO, Computer Vision, GIS, OCR, and multi-source data reconciliation.                                                     |
| validation-sources | Internal portal data, field imagery, GIS coordinates, Frontier database, AEP records, and workflow history.                |
| key-capabilities   | Pole detection, ownership verification, structural assessment, ambiguity detection, and confidence-based decision support. |
| decision-engine    | Produces outcomes such as Auto-Approve, Manual Review, or Reject / Rework based on fused evidence.                         |
| business-impact    | Reduces manual effort, improves validation consistency, and speeds up infrastructure approval workflows.                   |

| Attribute           | Description                                                                                               |
|---------------------|-----------------------------------------------------------------------------------------------------------|
| current-limitations | Small dataset, class imbalance, and low recall in the current detection setup.                            |
| future-improvements | Expand dataset, improve labeling quality, reduce class complexity, and strengthen active learning.        |
| human-in-the-loop   | Final decisions should remain reviewer-supported until the model reaches stronger validation reliability. |

## Post Files

```
src/content/posts/  
└─ pole-detection/  
   └─ cover.png  
   └─ index.mdx  
  
GitHub Repo: dranubhaparashar/Pole-Detection/  
└─ dataset/  
└─ runs/  
   └─ detect/  
└─ 001_png.rf.72b75e71c8e9e08119e23830becb46ea.jpg  
└─ 001_png.rf.72b75e71c8e9e08119e23830becb46ea.txt  
└─ 1.ipynb  
└─ 1.py  
└─ README.md  
└─ app.py  
└─ data.yaml  
└─ yolo26n.pt
```

## Why This Matters

**Note** Manual validation involves **13+ checks across multiple systems**.

**Important** Wrong validation → **cost impact + safety risks + delays**

**Tip** AI enables **confidence-based automated decisions**

**Warning** Low recall indicates **dataset limitation and class imbalance**

**Caution** Decisions should not be fully automated until model recall and validation coverage improve.

## What This System Does

flowchart TD

```
A[Internal Portal / Job Request] --> B[Candidate Pole Context]
B --> C[GIS Coordinates & Route Alignment]
B --> D[Field / Street / Inspection Imagery]
B --> E[Frontier Database]
B --> F[AEP / External Utility Records]
B --> G[OnPath / Workflow History]

D --> H[YOLO26n Detection]
D --> I[OCR / Tag Extraction]
C --> J[Geo-Matching Engine]
E --> K[Ownership & Structural Signals]
F --> K
G --> L[Design Context Signals]

H --> M[Evidence Fusion Layer]
I --> M
J --> M
K --> M
L --> M

M --> N[Rule Engine]
N --> O[Confidence Scoring Engine]
O --> P{Decision Engine}

P -->|High confidence| Q[Auto-Approve]
P -->|Conflicting evidence| R[Manual Review]
P -->|Low confidence| S[Reject / Rework]

R --> T[Reviewer Feedback]
T --> U[Audit Store]
T --> V[Threshold Tuning / Future Improvement]
```

## Core Capabilities

- Pole detection (YOLO26n)
- Ownership verification (AEP / Frontier DB)
- Structural assessment (visual + metadata)
- Space feasibility analysis
- Multi-source reconciliation
- Confidence-based decision engine

## Bento Overview

| Capability    | Description               |
|---------------|---------------------------|
| Pole Identity | Is this the correct pole? |
| Ownership     | Frontier or not?          |
| Ambiguity     | Nearby pole confusion     |
| Structure     | Safe for attachment?      |
| Decision      | Approve / Review / Reject |

## YOLO26n Detection Engine

### Model Details

- Layers: 260
- Parameters: 2.5M
- Classes: 36
- Framework: Ultralytics YOLO

### Training Setup

- Epochs: 100
- Batch: 16
- Image Size: 640

## Inference

```
from ultralytics import YOLO

model = YOLO("best.pt")

results = model("pole.jpg")
```

```
for r in results:
    print(r.bboxes)
```

---

## Confidence Engine

```
def confidence_score(pole, ownership, geo, structure):
    return (
        pole * 0.3 +
        ownership * 0.2 +
        geo * 0.3 +
        structure * 0.2
    )
```

---

## Decision Logic

```
def decision(score):
    if score > 0.85:
        return "Auto-Approve"
    elif score > 0.55:
        return "Manual Review"
    else:
        return "Reject"
```

---

## Example Threshold Update

```
- AUTO_APPROVE_THRESHOLD = 0.90
- MANUAL_REVIEW_THRESHOLD = 0.60
+ AUTO_APPROVE_THRESHOLD = 0.85
+ MANUAL_REVIEW_THRESHOLD = 0.55
  REJECT_THRESHOLD = 0.00
```

---

## Runtime Example

```
python train.py --model yolo26n.pt --data data.yaml --epochs 100 --batch 16 --imgsz 640
python infer.py --weights best.pt --source pole.jpg
python evaluate.py --predictions outputs/results.json
```

---

## Performance Snapshot

| Metric    | Value |
|-----------|-------|
| Precision | 0.608 |
| Recall    | 0.052 |
| mAP@0.5   | 0.054 |

**Warning** Low recall indicates **dataset limitation and class imbalance**

**Caution** This system should remain human-assisted until model recall and evidence consistency improve.

---

## Current Challenges

- Small dataset
- Class imbalance
- Low recall

---

## Next Improvements

- Increase dataset size
- Improve labeling
- Reduce class complexity
- Apply active learning

---

## Demo Videos

### Detection Demo

Embedded media: [Open video](#)

### Validation Demo

Embedded media: [Open video](#)

---

## Key Innovation

**Important** This is not just object detection —  
it is **Decision Intelligence for Infrastructure**

---

## Conclusion

This project transforms:

### Manual Validation → AI-Assisted Decision System

- Combines vision + data + reasoning.
  - Provides explainable decisions.
  - Enables faster approvals.
- 

## Final Thought

From **detecting poles** → to **understanding and deciding infrastructure**

The final recommendation is :spoiler[AI-assisted, not fully autonomous].